```python
a=[1,2,3,4,5]
print(a)
```

```
[1, 2, 3, 4, 5]
```

```python
import numpy as np
arr=np.array([1,2,3,4,5,6])
print(arr)
```

```
[1 2 3 4 5 6]
```

```python
l1=[1,2,3]
l2=[4,5,6]
addd=[]
for i in range(len(l1)):
    add=l1[i]+l2[i]
    addd.append(add)
print(addd)
```

```
[5, 7, 9]
```

```python
import numpy as np
a=np.array([1,2,3])
b=np.array([4,5,6])
print(a+b)
print(a-b)
print(a*b)
print(a/b)
print(a//b)
print(a%b)
print(a**b)
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
[0 0 0]
[1 2 3]
[  1  32 729]
```

```python
import numpy as np
arr1D=np.array([1,2,3,4,5])
arr2D=np.array([[1,2,3],[4,5,6]])
arr3D=np.array([[[1,2,3,0],[4,5,6,7],[8,9,10,11]]])
print(arr1D)
print(arr2D)
print(arr3D)
print(arr2D.shape)
print(arr3D.ndim)
print(arr2D.size)
print(arr3D.dtype)
```

```
[1 2 3 4 5]
[[1 2 3]
 [4 5 6]]
[[[ 1  2  3  0]
  [ 4  5  6  7]
  [ 8  9 10 11]]]
(2, 3)
3
6
int64
```

```
import numpy as np
a=np.array([1,2,3,4,5])
b=np.zeros(5)
print(a)
print(b)
c=np.zeros((2,2))
print(c)
d=np.ones(3)
print(d)
e=np.ones((2,3))
print(e)
```

```
[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[[0. 0.]
 [0. 0.]]
[1. 1. 1.]
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
import numpy as np
a=np.array([1,2,3,4,5])
b=np.zeros(5)
print(a)
print(b)
print(np.arange(1,10,2))
print(np.linspace(0,20,3))
print(np.random.rand(5))
print(np.random.randint(1,100,7))
print(np.random.randn(5))
```

```
[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[1 3 5 7 9]
[ 0. 10. 20.]
[0.1013688  0.14385852 0.09773514 0.35496593 0.32366422]
[58 29 45 82 65 82 62]
[ 0.25417744  0.79617542  1.30034906 -1.5196136   0.95799602]
```

```python
#Indexing and slicing
import numpy as np
a=np.array([10,20,30,40,50,60,70])
#1d array indexing
print(a[2])
print(a[-1])
b=np.array([[1,2,3],
            [4,5,6],
            [7,8,9]])
#2d array indexing
print(b[1][1])
#1d slicing
print(a[2:6])
print(a[:4])
print(a[::2])
print(a[::-1])

#2d slicing
print(b[1])
print(b[:,1])
print(b[:])

#boolean indexing
print(a[a>50])
print(a[a<50])
print(a[a%2==0])
#fancy indexing
print(a[[0,4,3,6]])
#modify values
a[4]=1000
print(a)
```

```
30
70
5
[30 40 50 60]
[10 20 30 40]
[10 30 50 70]
[70 60 50 40 30 20 10]
[4 5 6]
[2 5 8]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[60 70]
[10 20 30 40]
[10 20 30 40 50 60 70]
```

```
[10 50 40 70]
[  10   20   30   40 1000   60   70]
```

```python
#mathematical operation in numpy
#scalar broadcasting
import numpy as np
a=np.array([1,2,3])
print(a+10)
#2D broadcasting
b=np.array([[1,2,3],
            [4,5,6]])
print(b+10)
#row broadcasting
print(b[1]+5)
print(a+b)
```

```
[11 12 13]
[[11 12 13]
 [14 15 16]]
[ 9 10 11]
[[2 4 6]
 [5 7 9]]
```

```python
#matrix operation
import numpy as np
a=np.array([[1,2],
           [3,4]])
b=np.array([[4,3],
            [2,1]])
print(a*b)
print(np.dot(a,b))
print(a@b)
print(np.matmul(a,b))
print(a.T)  #row become column
```

```
[[4 6]
 [6 4]]
[[ 8  5]
 [20 13]]
[[ 8  5]
 [20 13]]
[[ 8  5]
 [20 13]]
[[1 3]
 [2 4]]
```

```python
# universal mathematical function
import numpy as np
a=np.array([4,9,16])
print(np.sqrt(a))
```

```python
b=np.array([-1,-2,-3,-4,-5])
print(np.abs(b))
print(np.log(a))
print(np.exp(a))
```

```
[2. 3. 4.]
[1 2 3 4 5]
[1.38629436 2.19722458 2.77258872]
[5.45981500e+01 8.10308393e+03 8.88611052e+06]
```

```python
#round function
arr=np.array([1.234,5.674])
print(np.ceil(arr))
print(np.floor(arr))
print(np.round(arr,2))
```

```
[2. 6.]
[1. 5.]
[1.23 5.67]
```

```python
#statistical function
import numpy as np
a=np.array([10,2,3,40,50])
print(np.mean(a))
print(np.median(a))
print(np.std(a))
print(np.max(a))
print(np.min(a))
print(np.percentile(a,25))
print(np.var(a))
print(np.argmin(a))
print(np.argmax(a))
b=np.array([1,2,3,4,5])
c=np.array([2,4,6,8,10])
print(np.corrcoef(a,b))
#aggregate Function
d=np.array([[1,3,4],
          [5,6,8]])
print(np.sum(d))
print(np.sum(d,axis=0))
print(np.sum(d,axis=1))
```

```
21.0
10.0
20.039960079800558
50
2
3.0
401.6
1
```

```
4
[[1.         0.83272222]
 [0.83272222 1.        ]]
27
[ 6  9 12]
[ 8 19]
```

Practice

```python
#create a 2D numpy array of students scores,compute class
average,highest scorer,normalize score to 0-1 range
import numpy as np
score=np.array([[50,60,70],
                [55,60,80],
                [67,80,90],
                [54,89,77]])
sum_indivi=np.sum(score,axis=1)
print(sum_indivi)
print(np.average(score))
print(np.max(sum_indivi))
print(np.min(sum_indivi))
total=np.sum(score)
normal=(score-score.min())/score.max()-score.min()
print(normal)
sub_total=np.sum(score,axis=0)
print(sub_total)
```

```
[180 195 237 220]
69.33333333333333
237
180
[[-50.         -49.88888889 -49.77777778]
 [-49.94444444 -49.88888889 -49.66666667]
 [-49.81111111 -49.66666667 -49.55555556]
 [-49.95555556 -49.56666667 -49.7       ]]
[226 289 317]
```

```python
#given arr=np.array([10,20,30,40,50]),write numpy expressions to get
all elements>25 and compute their sum
arr=np.array([10,20,30,40,50])
maxi=arr[arr>25]
print(maxi)
print(sum(maxi))
```

```
[30 40 50]
120
```